

VSCode + Gemini & LaTeX

環境構築ガイド

AI を活用した学習・研究環境の構築

目標 1 VSCode 上で Gemini を Agent として使えるようになる

目標 2 VSCode 上で LaTeX をコンパイルして PDF を出力できる
ようになる

実践例 電磁気学のマクスウェル方程式を PDF で出力する

2026 年度版

目次

1. はじめに	4
1.1. 全体の流れ	4
2. VSCode のインストール	5
2.1. ダウンロードとインストール	5
2.1.1. Windows	5
2.1.2. macOS	5
2.1.3. Linux (Ubuntu/Debian)	5
2.2. 初期設定	6
2.2.1. 日本語 UI のインストール	6
2.2.2. おすすめ設定	6
3. Gemini を Agent として使う	7
3.1. Gemini Code Assist とは	7
3.2. インストール手順	7
3.2.1. Step 1: 拡張機能のインストール	7
3.2.2. Step 2: Google アカウントでサインイン	7
3.2.3. Step 3: 動作確認	8
3.3. Gemini Chat の使い方	8
3.3.1. チャットパネルの操作	8
3.3.2. ファイルの参照	8
3.4. Agent モードの活用	8
3.4.1. Agent モードの起動	8
3.4.2. Agent モードの活用例	8
3.5. インライン補完の使い方	8
4. LaTeX 環境の構築（追加目標）	10
4.1. TeX Live のインストール	10
4.1.1. Windows	10
4.1.2. macOS	10
4.1.3. Linux (Ubuntu/Debian)	10
4.2. LaTeX Workshop 拡張機能のインストール	11
4.3. settings.json の設定	11
4.3.1. 設定ファイルを開く	11
4.3.2. 設定の追加	11
4.3.3. 設定項目の説明	12
4.4. .latexmkrc の設定	12
4.4.1. ファイルの配置場所	12
4.4.2. 設定内容	13
4.4.3. Windows でファイルを作成する方法	14

4.4.4. 動作確認	14
5. 例：マクスウェル方程式	16
5.1. マクスウェル方程式について	16
5.1.1. 微分形	16
5.1.2. 文書クラス	16
5.1.3. 使用する主なパッケージ	16
5.2. LaTeX ソースコード	16
5.3. コンパイルと PDF の確認	19
5.3.1. VSCode での手順	19
5.3.2. SyncTeX の使い方	19
5.3.3. よくあるエラーと対処法	19
6. 練習：Gemini Agent を使って LaTeX を書く	20
6.1. 課題	20
6.1.1. Gemini へのプロンプト例	20
6.2. 正解となる出力	20
6.2.1. アインシュタインテンソルの定義	20
6.2.2. アインシュタイン場の方程式（宇宙項あり）	20
6.2.3. 記号の説明	20
6.2.4. 確認ポイント	21
7. まとめ	22
7.1. 次のステップ	22
7.2. 参考リンク	22
A. LaTeX 数学表記リファレンス	23
A.1. 数学フォントスタイル	23
A.2. アクセント・デコレータ	23
A.2.1. 1 文字用アクセント	23
A.2.2. 複数文字・幅広アクセント	24
A.3. ギリシャ文字	24
A.4. よく使う数学記号	24
A.4.1. 関係演算子	24
A.4.2. 矢印	25
A.4.3. 二項演算子・その他記号	25
A.5. 数式環境の使い分け	25

1. はじめに

本資料は, VSCode (Visual Studio Code) を中心とした学習・研究環境を構築するためのステップバイステップガイドです.

達成目標:

1. メイン目標: VSCode 上で Google の AI 「Gemini」 を Agent として使えるようになる
2. 追加目標: VSCode 上で LaTeX をコンパイルし, 美しい PDF を出力できるようになる
3. 実践例: 電磁気学のマクスウェル方程式を LaTeX で記述し, PDF として出力する

各ステップを順に実施すれば必ず環境が整います. エラーが出てもしっかりと対処しましょう.

メモ 本資料は主に **Windows** を対象としています. macOS・Linux の場合は, 各節の該当箇所を読み替えてください.

1.1. 全体の流れ

手順	内容	所要時間の目安
1	VSCode のインストール	10 分
2	Gemini Code Assist の設定	15 分
3	TeX Live のインストール	30~120 分 (ダウンロードに依存)
4	LaTeX Workshop の設定	15 分
5	マクスウェル方程式の出力確認	10 分

2. VSCode のインストール

Visual Studio Code (VSCode) は Microsoft が開発した無料のエディタです。軽量でありながら拡張機能により LaTeX 編集・AI アシスタントなど多様な用途に使えます。

2.1. ダウンロードとインストール

2.1.1. Windows

1. ブラウザで以下の URL にアクセスする：

```
https://code.visualstudio.com/
```

2. 青い「Download for Windows」ボタンをクリックし、インストーラー (.exe) をダウンロードする
3. ダウンロードしたファイルをダブルクリックして実行する
4. インストールのオプション画面では、以下を必ずチェックする：
 - ・「PATH への追加（再起動後に使用可能）」
 - ・「エクスプローラーのファイルコンテキストメニューに [Code で開く] アクションを追加する」
5. 「インストール」ボタンを押して完了を待つ

注意 「PATH への追加」を忘れると、ターミナルから `code` コマンドで VSCode を起動できません。インストール後にターミナルで `code --version` を実行して確認してください。

2.1.2. macOS

1. <https://code.visualstudio.com/> からダウンロードする
 - ・ M1/M2/M3 Mac (Apple Silicon) は「Apple Silicon」版を選ぶ
 - ・ Intel Mac は「Intel chip」版を選ぶ
2. ダウンロードした .zip を展開し、Visual Studio Code.app を「アプリケーション」フォルダに移動する
3. VSCode を起動し、コマンドパレット (**Shift+Cmd+P**) を開く
4. 「Shell Command: Install 'code' command in PATH」と入力して実行する

2.1.3. Linux (Ubuntu/Debian)

Microsoft のリポジトリを追加して VSCode をインストール

```
sudo apt-get install -y wget gpg
wget -q0- https://packages.microsoft.com/keys/microsoft.asc \
  | gpg --dearmor > packages.microsoft.gpg
sudo install -D -o root -g root -m 644 \
  packages.microsoft.gpg /etc/apt/keyrings/packages.microsoft.gpg
echo "deb [arch=amd64 signed-by=/etc/apt/keyrings/packages.microsoft.gpg] \
  https://packages.microsoft.com/repos/code stable main" \
  | sudo tee /etc/apt/sources.list.d/vscode.list > /dev/null
sudo apt update && sudo apt install code
```

2.2. 初期設定

2.2.1. 日本語 UI のインストール

1. 拡張機能パネル (`Ctrl+Shift+X`) を開く
2. 検索欄に「Japanese Language Pack」と入力する
3. Microsoft 製「Japanese Language Pack for Visual Studio Code」をインストールする
4. 再起動すると UI が日本語になる

2.2.2. おすすめ設定

設定ファイル (`Ctrl+,`) から以下を変更することを推奨します：

設定項目	推奨値
Editor: Font Family	'Consolas', 'Courier New', monospace
Editor: Font Size	14
Editor: Tab Size	2 または 4
Editor: Word Wrap	on (折り返し表示)
Files: Auto Save	afterDelay (自動保存)

3. Gemini を Agent として使う

Google の AI「Gemini」を VSCode で使うには、**Gemini Code Assist** 拡張機能を導入します。これは Google が公式に提供する無料のコーディング支援ツールです。

3.1. Gemini Code Assist とは

Gemini Code Assist は、Gemini モデルを搭載した AI プログラミングアシスタントです。主な機能は以下の通りです：

機能	説明
コード補完	入力中にリアルタイムでコードを提案。Tab で受け入れ
チャット	コードについて質問・コード生成を自然言語で依頼
Agent モード	複数ファイルの読み書き・コマンド実行など自律的にタスクを実行
インライン編集	選択範囲を指定して部分的な修正を依頼

無料枠（個人利用、Google アカウント必要）：

- コード補完: 月 6,000 回
- チャット: 月 240 回

3.2. インストール手順

3.2.1. Step 1: 拡張機能のインストール

1. VSCode の拡張機能パネル (**Ctrl+Shift+X**) を開く
2. 検索欄に「Gemini Code Assist」と入力する
3. 発行元が Google であることを確認して「インストール」をクリックする

メモ 似た名前の拡張機能が複数存在します。必ず「Google」が発行元のものを選んでください。

3.2.2. Step 2: Google アカウントでサインイン

1. インストール後、VSCode の左サイドバーに Gemini のアイコン（星形）が表示される
2. アイコンをクリックするか、コマンドパレット (**Ctrl+Shift+P**) から「Gemini: Sign In」を実行する
3. ブラウザが開くので、Google アカウントでログインする
4. アクセス許可を求められたら「許可」をクリックする
5. VSCode に戻るとサインイン完了

注意 大学メールアドレス (@ms.saitama-u.ac.jp) でサインインすると、組織のポリシーにより機能が制限される場合があります。個人の Gmail アカウントの使用を推奨します。

3.2.3. Step 3: 動作確認

サインイン後、左サイドバーの Gemini アイコンをクリックしてチャットパネルを開き、以下のメッセージを送信してみましょう：

こんにちは！ VSCode 上で Gemini が使えるか確認しています。

Gemini から返答が来れば設定完了です。

3.3. Gemini Chat の使い方

3.3.1. チャットパネルの操作

チャットパネルはサイドバーの Gemini アイコンから開きます。または `Ctrl+Shift+P` → 「Gemini: Open Chat」でも開けます。

入力例（自然な日本語で OK）

このコードの動作を説明してください

以下のエラーの解決方法を教えてください：

[エラーメッセージを貼り付ける]

Python で CSV ファイルを読み込んで平均値を計算するコードを書いてください

3.3.2. ファイルの参照

チャット入力欄で @ を入力すると、プロジェクト内のファイルを参照できます：

@maxwell.tex このファイルにある数式の意味を説明してください

3.4. Agent モードの活用

Agent モードでは、Gemini が複数のステップを自律的に実行します。ファイルの読み書き・コマンド実行などもできます。

3.4.1. Agent モードの起動

チャットパネル下部の「Agent Preview」を on にします。

3.4.2. Agent モードの活用例

プロジェクト内のすべての .tex ファイルを確認して、
使用しているパッケージの一覧をまとめてください

maxwell.tex を開いて、数式の番号を付け直してください

README.md を日本語で作成してください

注意 Agent モードはファイルを自動編集します。大切なファイルはあらかじめバックアップを取るか、Git でバージョン管理することを強く推奨します。（Git についてはここでは説明しませんが、知りたい人は AI に聞いてみてください。というか、Git は AI に操作させた方が良くかもしれません。）

3.5. インライン補完の使い方

コードを書いているとき、Gemini が補完候補を薄いグレーで表示します：

Tab	候補を受け入れる
Esc	候補を無視する
Ctrl+Enter	複数の候補を表示する
Alt+]	次の候補を表示する

4. LaTeX 環境の構築（追加目標）

LaTeX は論文・技術文書を美しくタイプセットするためのシステムです。理工系の学術論文では事実上の標準となっています。

4.1. TeX Live のインストール

TeX Live は LaTeX の実行環境（処理系）です。まずこれをインストールします。

4.1.1. Windows

4.1.1.1. 公式インストーラーを使う方法（推奨）

1. 以下にアクセスする：

<https://www.tug.org/texlive/>

2. 「download」をクリックし、「install-tl-windows.exe」をダウンロードする
3. ダウンロードしたファイルを管理者として実行する
4. インストーラーが起動したら「Install」をクリックして待つ

注意 フルインストールには約 7～8 GB のディスク容量と 1～2 時間かかります。ネットワーク環境の良い場所（学内 Wi-Fi など）で行ってください。途中でキャンセルしないでください。

インストール後、コマンドプロンプト（Win+R → cmd）で確認します：

```
lualatex --version
```

バージョン情報が表示されれば成功です。

4.1.1.2. winget を使う方法

```
winget install --id TeXLive.TeXLive
```

4.1.2. macOS

MacTeX（macOS 向け TeX Live）をインストールします：

1. <https://www.tug.org/mactex/> にアクセスする
2. 「MacTeX Download」をクリックし、.pkg（約 4GB）をダウンロードする
3. ダウンロードしたファイルをダブルクリックしてインストールする

または Homebrew を使う場合：

```
brew install --cask mactex-no-gui
```

4.1.3. Linux (Ubuntu/Debian)

フルインストール（推奨、約 5GB）

```
sudo apt-get install texlive-full
```

日本語のみ必要な場合（最小構成）

```
sudo apt-get install texlive-lang-japanese \
    texlive-latex-extra texlive-fonts-recommended
```

4.2. LaTeX Workshop 拡張機能のインストール

LaTeX Workshop は VSCode で LaTeX を書くための最も人気のある拡張機能です。

1. 拡張機能パネル (`Ctrl+Shift+X`) を開く
2. 「LaTeX Workshop」と検索する
3. James Yu 氏の「LaTeX Workshop」をインストールする

主な機能：

- 保存時に自動コンパイル
- PDF プレビュー (VSCode 内で表示)
- シンタックスハイライトとコマンド補完
- SyncTeX (PDF とソースの双方向ジャンプ)
- エラーメッセージの可視化

4.3. settings.json の設定

VSCode の設定ファイル `settings.json` を編集して、LaTeX Workshop の動作を設定します。

4.3.1. 設定ファイルを開く

1. `Ctrl+Shift+P` でコマンドパレットを開く
2. Preferences: Open User Settings (JSON) と入力して選択する
3. `settings.json` が開く

4.3.2. 設定の追加

以下の内容を `settings.json` に追加してください (既存の `{...}` の内部に追記します)：

```
{
  // === LaTeX Workshop 設定 ===

  "latex-workshop.latex.tools": [
    {
      "name": "latexmk",
      "command": "latexmk",
      "args": [
        "-synctex=1",
        "-interaction=nonstopmode",
        "-file-line-error",
        "%DOC%"
      ],
      "env": {}
    }
  ],

  "latex-workshop.latex.recipes": [
    {
      "name": "latexmk",
      "tools": ["latexmk"]
    }
  ]
}
```

```

],

"latex-workshop.view.pdf.viewer": "tab",

"latex-workshop.latex.autoBuild.run": "onSave",

"latex-workshop.synctex.afterBuild.enabled": true,

"latex-workshop.latex.autoClean.run": "onFailed",

"latex-workshop.latex.clean.fileTypes": [
  "*.aux", "*.bbl", "*.blg", "*.idx", "*.ind",
  "*.lof", "*.lot", "*.out", "*.toc",
  "*.fls", "*.log", "*.fdb_latexmk",
  "*.snm", "*.nav", "*.dvi", "*.synctex.gz"
]
}

```

ヒント settings.json に既に設定がある場合は、既存の { と } の内側に追記してください。JSON では、最後の要素の後ろにカンマを付けてはいけません。

4.3.3. 設定項目の説明

設定名	説明
latex.tools	コンパイルに使うコマンドを定義する
latex.recipes	ビルドレシピ（ツールの組み合わせ）を定義する
view.pdf.viewer	PDF を VSCode のタブ内で表示する
autoBuild.run	ファイル保存時に自動コンパイルする
synctex.afterBuild.enabled	コンパイル後に PDF の対応箇所に移動する
autoClean.run	ビルド失敗時に中間ファイルを削除する

4.4. .latexmkrc の設定

.latexmkrc は latexmk コマンドの動作を設定するファイルです。使用する処理系 (LuaLaTeX / upLaTeX) をファイル先頭のコメントや 文書クラスから自動判定します。

4.4.1. ファイルの配置場所

.latexmkrc は以下のどちらかに配置します：

- プロジェクトフォルダ内: そのフォルダのみに適用
- ホームディレクトリ: 全プロジェクトに適用
 - Windows: C:\Users\ユーザー名\.latexmkrc
 - macOS/Linux: ~/.latexmkrc

4.4.2. 設定内容

以下の内容で `.latexmkrc` を作成してください：

```
#!/usr/bin/env perl

# 文書側の指定を優先し、なければ documentclass からエンジンを推定する。
# % !TEX program = lualatex
# % !TEX program = uplatex
my $texfile = find_tex_file();
my $engine = detect_engine($texfile);

if ($engine eq 'uplatex' || $engine eq 'platex') {
    # pLaTeX / upLaTeX で DVI を作成し、dvi2pdf で PDF 化する (pdf_mode = 3)
    $pdf_mode = 3;
    $latex = "$engine -synctex=1 -halt-on-error "
        . "-interaction=nonstopmode %0 %S";
    $dvi2pdf = 'dvi2pdf %0 -o %D %S';
} else {
    # LuaLaTeX で直接 PDF を生成する (pdf_mode = 4)
    $pdf_mode = 4;
    $lualatex = 'lualatex -synctex=1 -halt-on-error '
        . '-interaction=nonstopmode %0 %S';
}

# 参考文献の処理
$bibtex = 'upbibtex %0 %B';

# 索引の処理 (日本語対応)
$makeindex = 'mendex %0 -o %D %S';

# クリーンアップ対象ファイル
$clean_full_ext = '%R.synctex.gz %R.auxlock';

sub find_tex_file {
    for my $arg (@ARGV) {
        next if $arg =~ /^-/;
        return $arg if $arg =~ /\.tex$/i && -f $arg;
        my $candidate = "$arg.tex";
        return $candidate if -f $candidate;
    }
    return undef;
}

sub detect_engine {
    my ($file) = @_;
    return 'lualatex' unless defined $file && -f $file;
    open(my $fh, '<:raw', $file) or return 'lualatex';
    my $content = do { local $/; <$fh> };
}
```

```

close($fh);
if ($content =~ /^%!\s*\s*!TEX\s+program\s*=\s*(lualatex|uplatex|platex)\s*$/mi) {
    return lc($1);
}
if ($content =~ /\s*\documentclass(?:\s*[\^\\]*\s*)?{\s*(?:jsarticle|jsbook|jsreport|
jarticle|jbook|tarticle|tbook)\s*}/) {
    return 'uplatex';
}
return 'lualatex';
}

```

メモ エンジンの選択ルール（優先順）：

1. .tex ファイル先頭の % !TEX program = lualatex などのコメント
2. jsarticle など旧来の日本語クラスを使っていれば upLaTeX
3. それ以外はデフォルトで LuaLaTeX

4.4.3. Windows でファイルを作成する方法

VSCode のターミナル (**Ctrl+@**) を開いて以下を実行すると確実です：

```

# PowerShell: ホームディレクトリに .latexmkrc を作成して開く
New-Item -Path "$HOME\.latexmkrc" -ItemType File -Force
code "$HOME\.latexmkrc"

```

4.4.4. 動作確認

以下の内容で test.tex を作成して動作確認します：

```
\documentclass[lualatex, a4paper, 12pt]{jlrreq}
```

```
\title{動作確認}
```

```
\author{自分の名前}
```

```
\date{\today}
```

```
\begin{document}
```

```
\maketitle
```

```
\section{はじめに}
```

LaTeX の動作確認用ファイルです。

```
\begin{equation}
```

```
E = mc^2
```

```
\end{equation}
```

```
\end{document}
```

ファイルを保存 (**Ctrl+S**) すると自動でコンパイルが始まります。右上の PDF アイコン（または **Ctrl+Alt+V**）をクリックして PDF を確認します。

ヒント コンパイルのログは「出力」パネル (`Ctrl+Shift+U`) の「LaTeX Workshop」を選択すると確認できます。エラーが出た場合は必ずここを見てください。

5. 例：マクスウェル方程式

電磁気学の基本方程式であるマクスウェル方程式を LaTeX で記述し、PDF として出力します。

5.1. マクスウェル方程式について

マクスウェル方程式は電磁気学を支配する 4 つの方程式の組です。電場 $\mathbf{E}$ 、磁場 $\mathbf{B}$ 、電荷密度 ρ 、電流密度 $\mathbf{J}$ の関係を記述します。

5.1.1. 微分形

$$\begin{aligned}\nabla \cdot \mathbf{E} &= \frac{\rho}{\epsilon_0} & \nabla \cdot \mathbf{B} &= 0 \\ \nabla \times \mathbf{E} &= -\frac{\partial \mathbf{B}}{\partial t} & \nabla \times \mathbf{B} &= \mu_0 \mathbf{J} + \mu_0 \epsilon_0 \frac{\partial \mathbf{E}}{\partial t}\end{aligned}$$

5.1.2. 文書クラス

文書クラスは `\documentclass[オプション]{クラス名}` で指定し、文書全体の体裁を決める。

クラス名	対応処理系	説明
jlreq	LuaLaTeX / upLaTeX	日本語組版規則 (JIS X 4051) に準拠したモダンなクラス。本資料で使用
jsarticle/ jsbook	upLaTeX / pLaTeX	日本で広く使われてきた標準クラス。レポート・書籍向け
article/ book	pdfLaTeX / LuaLaTeX	LaTeX 標準クラス。英文向けだが日本語も使用可
beamer	LuaLaTeX など	プレゼンテーションスライド作成用クラス

メモ jlreq はオプションに処理系 (lualatex など) を指定するだけで日本語が使える。日本語組版の新しい文書には jlreq を使うことを推奨する。

5.1.3. 使用する主なパッケージ

パッケージ	用途
amsmath	高度な数式環境 (align 環境など)
amssymb	数学記号の拡張
bm	太字のベクトル記号 <code>\bm{E}</code> など
physics	物理向けコマンド (<code>\div</code> , <code>\curl</code> など)

5.2. LaTeX ソースコード

以下の内容を `maxwell.tex` として保存してください (`examples/maxwell.tex` として保存することを推奨します)：

```
% マクスウェル方程式 --- LuaLaTeX でコンパイル
\documentclass[lualatex, a4paper, 12pt]{jlreq}
```



```

% 数学パッケージ
\usepackage{amsmath}      % 高度な数式環境
\usepackage{amssymb}      % 数学記号
\usepackage{bm}           % 太字ベクトル \bm{...}

% ページ設定（オプション）
\usepackage{geometry}
\geometry{top=25mm, bottom=25mm, left=25mm, right=25mm}

\title{マクスウェル方程式}
\author{電磁気学}
\date{\today}

\begin{document}
\maketitle

\section{概要}

電磁気学はマクスウェル方程式と呼ばれる4つの方程式によって
完全に記述される。
電場  $\bm{E}$ , 磁場  $\bm{B}$ , 電荷密度  $\rho$ ,
電流密度  $\bm{J}$  を用いて表される。

\section{積分形}

\begin{align}
\oint_S \bm{E} \cdot d\bm{A}
&= \frac{Q}{\epsilon_0} \\
\tag{ガウスの法則} \\
\label{eq:gauss_e} \\
\\[8pt]
\oint_S \bm{B} \cdot d\bm{A}
&= 0 \\
\tag{磁気のガウスの法則} \\
\label{eq:gauss_b} \\
\\[8pt]
\oint_C \bm{E} \cdot d\bm{l}
&= -\frac{d}{dt} \oint_S \bm{B} \cdot d\bm{A} \\
\tag{ファラデーの法則} \\
\label{eq:faraday} \\
\\[8pt]
\oint_C \bm{B} \cdot d\bm{l}
&= \mu_0 \oint_S \bm{J} \cdot d\bm{A}
+ \mu_0 \epsilon_0 \frac{d}{dt} \oint_S \bm{E} \cdot d\bm{A} \\
\tag{アンペール--マクスウェルの法則} \\
\label{eq:ampere}

```

```
\end{align}
```

```
\section{微分形}
```

ガウスの定理とストークスの定理を用いると、
積分形から微分形が導かれる。

```
\begin{align}
  \nabla \cdot \bm{E}
    &= \frac{\rho}{\varepsilon_0}
    \tag{ガウスの法則}
    \label{eq:gauss_e_diff}
    \\[8pt]
  \nabla \cdot \bm{B}
    &= 0
    \tag{磁気のガウスの法則}
    \label{eq:gauss_b_diff}
    \\[8pt]
  \nabla \times \bm{E}
    &= -\frac{\partial \bm{B}}{\partial t}
    \tag{ファラデーの法則}
    \label{eq:faraday_diff}
    \\[8pt]
  \nabla \times \bm{B}
    &= \mu_0 \bm{J}
      + \mu_0 \varepsilon_0 \frac{\partial \bm{E}}{\partial t}
    \tag{アンペール--マクスウェルの法則}
    \label{eq:ampere_diff}
\end{align}
```

```
\section{物理定数}
```

```
\begin{table}[h]
  \centering
  \caption{電磁気学の基本定数}
  \begin{tabular}{ccl}
    \hline
    記号 & 名称 & 値 \\
    \hline
     $\varepsilon_0$  & 真空の誘電率 &  $8.854 \times 10^{-12} \mathrm{F/m}$  \\
     $\mu_0$  & 真空の透磁率 &  $4\pi \times 10^{-7} \mathrm{H/m}$  \\
     $c$  & 真空中の光速 &  $c = 1/\sqrt{\mu_0 \varepsilon_0}$   

 $\approx 3.0 \times 10^8 \mathrm{m/s}$ 
    \hline
  \end{tabular}
\end{table}
```

```
\label{tab:constants}
\end{table}
```

```
\section{電磁波}
```

マクスウェル方程式から、電場と磁場が波として伝播すること
(電磁波) が導かれる。電場の波動方程式:

```
\begin{equation}
\nabla^2 \bm{E}
- \mu_0 \varepsilon_0 \frac{\partial^2 \bm{E}}{\partial t^2}
= \bm{0}
\end{equation}
```

この波の速度は $c = 1/\sqrt{\mu_0 \varepsilon_0}$ であり、
光速と一致する。これがマクスウェルによる
「光は電磁波である」という発見である。

```
\end{document}
```

5.3. コンパイルと PDF の確認

5.3.1. VSCode での手順

1. maxwell.tex を VSCode で開く
2. **Ctrl+S** で保存すると自動コンパイルが始まる
3. 右上の PDF アイコンをクリック (または **Ctrl+Alt+V**) して PDF を確認する

5.3.2. SyncTeX の使い方

PDF と LaTeX ソースを行き来できます:

PDF 上で Ctrl+クリック	→ 対応するソース行に移動
ソース上で Ctrl+Alt+J	→ PDF の対応箇所に移動

5.3.3. よくあるエラーと対処法

エラー内容	対処法
package not found	tlmgr install パッケージ名 でパッケージを追加
Undefined control sequence	コマンドのスペルを確認。必要なパッケージが \usepackage されているか確認
Missing \ \$ inserted	数式環境 ($\dots$ や equation) の対応が正しいか確認
Font not found	TeX Live を再インストール, またはフォント名を変更
File not found	ファイルパスを確認。拡張子も含めて正確に指定

ヒント Gemini に「以下のエラーを解決してください: [エラーメッセージ]」と貼り付けると、具体的な解決策を提案してくれます!

6. 練習：Gemini Agent を使って LaTeX を書く

6.1. 課題

Gemini Agent に以下のプロンプトを与えて、**アインシュタイン方程式**の LaTeX ファイルを生成させよ。生成されたコードをコンパイルし、後述する「正解の出力」と見比べて確認すること。

6.1.1. Gemini へのプロンプト例

一般相対性理論のアインシュタイン場の方程式を LaTeX で書いてください。
条件は以下の通りです：

- `\documentclass[lualatex, a4paper, 12pt]{jllreq}` を使う
- `\usepackage{amsmath}`, `\usepackage{amssymb}`, `\usepackage{bm}` を使う
- 宇宙項 (Λ) を含む形で書く
- アインシュタインテンソルの定義式も書く
- 各記号 ($G_{\mu\nu}$, $R_{\mu\nu}$, R , $g_{\mu\nu}$, $T_{\mu\nu}$, Λ , G , c) の説明を表か簡条書きでまとめる
- `align` 環境を使って数式番号を付ける

ヒント プロンプトが具体的であるほど、Gemini の出力が正確になります。条件を簡条書きで整理して伝えるのが効果的です。

6.2. 正解となる出力

以下の数式と記号説明が PDF に表示されれば正解です。

6.2.1. アインシュタインテンソルの定義

$$G_{\mu\nu} = R_{\mu\nu} - \frac{1}{2}Rg_{\mu\nu}$$

6.2.2. アインシュタイン場の方程式（宇宙項あり）

$$G_{\mu\nu} + \Lambda g_{\mu\nu} = \frac{8\pi G}{c^4} T_{\mu\nu}$$

6.2.3. 記号の説明

記号	意味
$G_{\mu\nu}$	アインシュタインテンソル（時空の曲率を表す）
$R_{\mu\nu}$	リッチテンソル（曲率の縮約）
R	リッチスカラー（スカラー曲率）
$g_{\mu\nu}$	計量テンソル（時空の距離構造を定める）
$T_{\mu\nu}$	エネルギー運動量テンソル（物質・エネルギーの分布）
Λ	宇宙項（宇宙定数，真空エネルギーに対応）
G	ニュートンの重力定数 $G \approx 6.674 \times 10^{-11} \text{ N m}^2 / \text{kg}^2$
c	真空中の光速 $c \approx 3.0 \times 10^8 \text{ m/s}$

6.2.4. 確認ポイント

以下をすべて満たしていれば合格です：

1. アインシュタインテンソルの定義式 $G_{\mu\nu} = R_{\mu\nu} - \frac{1}{2}Rg_{\mu\nu}$ が出力されている
2. 場の方程式 $G_{\mu\nu} + \Lambda g_{\mu\nu} = 8\pi G/c^4 \cdot T_{\mu\nu}$ が出力されている
3. 各数式に番号が付いている (`align` 環境を使用)
4. 記号の説明が表または箇条書きで記載されている
5. エラーなしでコンパイルできる

メモ Gemini が生成したコードがそのままコンパイルできない場合は、Agent モードであればエラーメッセージを自動で読み取って修正まで行ってくれます。Agent モードを使っていない場合は、エラーメッセージをチャットに貼り付けて修正を依頼しましょう。

7. まとめ

本資料で構築した環境のまとめです：

ツール	用途	状態
VSCode	エディタ本体・すべての基盤	完了
日本語パック	UI を日本語化	完了
Gemini Code Assist	AI コーディング支援・Agent	完了
TeX Live	LaTeX 処理系（LuaLaTeX 含む）	完了
LaTeX Workshop	VSCode で LaTeX を快適に書く拡張機能	完了
settings.json	LaTeX Workshop の自動コンパイル設定	完了
.latexmkrc	LuaLaTeX を使うための latexmk 設定	完了
maxwell.tex	マクスウェル方程式の PDF 出力確認	完了

7.1. 次のステップ

環境が整ったら、以下にチャレンジしてみましょう：

1. Gemini × LaTeX: チャットで「アンペールの法則を LaTeX の align 環境で書いて」と依頼してみる
2. レポートを LaTeX で: 授業のレポートを LaTeX で書いてみる
3. 参考文献管理: biblatex + biber で参考文献リストを自動生成する
4. 図の挿入: graphicx パッケージで画像や PDF を文書に挿入する
5. Beamer でスライド作成: LaTeX でプレゼン資料を作る
6. TikZ で図の作成を試してみる

7.2. 参考リンク

TeX Wiki	https://texwiki.texjp.org/ 日本語の LaTeX 情報が充実
Overleaf	https://www.overleaf.com/ ブラウザで使えるオンライン LaTeX (環境構築不要)
TeX Live 公式	https://www.tug.org/texlive/
jlreq クラス	https://github.com/abenori/jlreq 本資料で使った文書クラス

メモ エラーが出て困ったときは、エラーメッセージをそのまま Gemini に貼り付けて聞いてみてください。また、TA にも気軽に相談してください。

Happy TeXing & Coding!

A. LaTeX 数学表記リファレンス

LaTeX で数式を書く際によく使う記法をまとめる. コンパイルには断りがない限り `amsmath`・`amssymb`・`bm` を `\usepackage` しておく必要がある.

A.1. 数学フォントスタイル

文字の形 (書体) を変えるコマンドの一覧. 引数には英字 1 文字でなく文字列も指定できる (例: `\mathbf{ABC}`).

LaTeX コマンド	出力例	必要パッケージ	主な用途
<code>\mathrm{A}</code>	A	(なし)	単位・演算子名 (<code>\sin</code> , <code>dx</code> など)
<code>\mathit{A}</code>	<i>A</i>	(なし)	数式のデフォルト書体 (通常は不要)
<code>\mathbf{A}</code>	A	(なし)	太字立体 (行列・ベクトルの一部表記)
<code>\boldsymbol{A}</code>	<i>A</i>	<code>amsmath</code>	太字イタリック (ベクトル・行列の標準表記)
<code>\bm{A}</code>	<i>A</i>	<code>bm</code>	<code>\boldsymbol</code> の改良版. ギリシャ文字にも有効
<code>\mathcal{A}</code>	<i>A</i>	(なし)	筆記体 (ラグランジアン $\mathcal{L}$, 集合族など)
<code>\mathbb{R}</code>	$\mathbb{R}$	<code>amssymb</code>	黒板太字 (実数 $\mathbb{R}$, 複素数 $\mathbb{C}$, 整数 $\mathbb{Z}$ など)
<code>\mathsf{A}</code>	A	(なし)	サンセリフ体 (行列表記の一流儀)
<code>\mathtt{A}</code>	A	(なし)	等幅 (コードや特定の定数)
<code>\mathfrak{g}</code>	$\mathfrak{g}$	<code>amssymb</code>	フラクトゥール体 (リー代数 $\mathfrak{g}$, $\mathfrak{su}(2)$ など)

ヒント 物理のベクトルには `\bm{a}` が最も一般的 (太字イタリック $\mathbf{a}$).
 数学の集合 $\mathbb{R}$ や演算子名 $\sin$ など, 分野慣習に合わせて使い分ける.

A.2. アクセント・デコレータ

文字の上下に記号を付けるコマンドの一覧. `{...}` 内には複数文字も入れられる.

A.2.1. 1 文字用アクセント

LaTeX コマンド	出力例	主な用途
<code>\hat{a}</code>	$\hat{a}$	推定量, 単位ベクトル ($\hat{e}$, $\hat{x}$)
<code>\tilde{a}</code>	$\tilde{a}$	フーリエ変換, 等価量 ($\tilde{f}$)
<code>\bar{a}</code>	$\bar{a}$	平均値, 複素共役 ($\bar{z}$)
<code>\vec{a}</code>	$\vec{a}$	ベクトル (手書き流儀; 物理では <code>\bm</code> が多い)
<code>\dot{a}</code>	$\dot{a}$	時間の 1 階微分 ($\dot{x} = dx/dt$)
<code>\ddot{a}</code>	$\ddot{a}$	時間の 2 階微分 ($\ddot{x} = d^2x/dt^2$)
<code>\acute{a}</code>	$\acute{a}$	鋭アクセント
<code>\grave{a}</code>	$\grave{a}$	重アクセント
<code>\breve{a}</code>	$\breve{a}$	ブレーブ
<code>\check{a}</code>	$\check{a}$	キャロン (チェック)
<code>\mathring{a}</code>	$\mathring{a}$	リング (ストロームなど)

A.2.2. 複数文字・幅広アクセント

LaTeX コマンド	出力例	説明
<code>\widehat{ABC}</code>	$\widehat{ABC}$	コンテンツ幅に合わせたハット
<code>\widetilde{ABC}</code>	$\widetilde{ABC}$	コンテンツ幅に合わせたチルダ
<code>\overline{ABC}</code>	$\overline{ABC}$	コンテンツ全体に上線
<code>\underline{ABC}</code>	$\underline{ABC}$	コンテンツ全体に下線
<code>\overbrace{A+B}</code>	$\overbrace{A+B}$	上ブレース
<code>\underbrace{A+B}</code>	$\underbrace{A+B}$	下ブレース

A.3. ギリシャ文字

大文字は先頭を大文字にする（例：`\Gamma`）。直立体が必要な場合は `\mathrm{\Gamma}` とする。

コマンド	出力	コマンド	出力	コマンド	出力
<code>\alpha</code>	α	<code>\iota</code>	ι	<code>\rho</code>	ρ
<code>\beta</code>	β	<code>\kappa</code>	κ	<code>\sigma</code>	σ
<code>\gamma</code>	γ	<code>\lambda</code>	λ	<code>\tau</code>	τ
<code>\delta</code>	δ	<code>\mu</code>	μ	<code>\upsilon</code>	υ
<code>\epsilon</code>	ϵ	<code>\nu</code>	ν	<code>\phi</code>	ϕ
<code>\zeta</code>	ζ	<code>\xi</code>	ξ	<code>\chi</code>	χ
<code>\eta</code>	η	<code>\pi</code>	π	<code>\psi</code>	ψ
<code>\theta</code>	θ	<code>\varpi</code>	ϖ	<code>\omega</code>	ω

コマンド（大文字）	出力	コマンド（大文字）	出力	コマンド（大文字）	出力
<code>\Gamma</code>	Γ	<code>\Lambda</code>	Λ	<code>\Phi</code>	Φ
<code>\Delta</code>	Δ	<code>\Xi</code>	Ξ	<code>\Chi</code>	$\Chi$
<code>\Theta</code>	Θ	<code>\Pi</code>	Π	<code>\Psi</code>	Ψ
<code>\Lambda</code>	Λ	<code>\Sigma</code>	Σ	<code>\Omega</code>	Ω
<code>\Upsilon</code>	Υ	<code>\Phi</code>	Φ		

メモ `\varepsilon` (ϵ), `\varphi` (ϕ), `\vartheta` (ϑ), `\varsigma` (ς) など、バリエーション形（var 付き）も存在する。物理では ϵ_0 （真空の誘電率）のように慣習的に使われる場合がある。

A.4. よく使う数学記号

A.4.1. 関係演算子

コマンド	出力	コマンド	出力	コマンド	出力
<code>\leq</code>	$\leq$	<code>\geq</code>	$\geq$	<code>\neq</code>	$\neq$
<code>\ll</code>	$\ll$	<code>\gg</code>	$\gg$	<code>\approx</code>	$\approx$
<code>\sim</code>	$\sim$	<code>\simeq</code>	$\simeq$	<code>\equiv</code>	$\equiv$

<code>\propto</code>	$\propto$	<code>\in</code>	$\in$	<code>\notin</code>	$\notin$
<code>\subset</code>	$\subset$	<code>\supset</code>	$\supset$	<code>\subseteq</code>	$\subseteq$

A.4.2. 矢印

コマンド	出力	コマンド	出力
<code>\to / \rightarrow</code>	$\rightarrow$	<code>\leftarrow</code>	$\leftarrow$
<code>\Rightarrow</code>	$\Rightarrow$	<code>\Leftarrow</code>	$\Leftarrow$
<code>\Leftrightarrow</code>	$\Leftrightarrow$	<code>\leftrightarrow</code>	$\leftrightarrow$
<code>\mapsto</code>	$\mapsto$	<code>\longrightarrow</code>	$\longrightarrow$
<code>\uparrow</code>	$\uparrow$	<code>\downarrow</code>	$\downarrow$
<code>\nearrow</code>	$\nearrow$	<code>\searrow</code>	$\searrow$

A.4.3. 二項演算子・その他記号

コマンド	出力	コマンド	出力	コマンド	出力
<code>\times</code>	$\times$	<code>\div</code>	$\div$	<code>\pm</code>	$\pm$
<code>\cdot</code>	$\cdot$	<code>\cdots</code>	$\cdots$	<code>\ldots</code>	$\ldots$
<code>\nabla</code>	∇	<code>\partial</code>	∂	<code>\infty</code>	∞
<code>\forall</code>	$\forall$	<code>\exists</code>	$\exists$	<code>\emptyset</code>	$\emptyset$
<code>\sum</code>	$\sum$	<code>\prod</code>	$\prod$	<code>\int</code>	$\int$
<code>\oint</code>	$\oint$	<code>\iint</code>	$\iint$	<code>\iiint</code>	$\iiint$
<code>\hbar</code>	$\hbar$	<code>\ell</code>	ℓ	<code>\Re</code>	$\Re$
<code>\Im</code>	$\Im$	<code>\dagger</code>	$\dagger$	<code>\star</code>	$\star$

A.5. 数式環境の使い分け

環境名	用途	必要パッケージ
<code>equation</code>	番号付き 1 行数式（標準）	（なし）
<code>equation*</code>	番号なし 1 行数式	<code>amsmath</code>
<code>align</code>	複数行, & で揃える, 各行に番号	<code>amsmath</code>
<code>align*</code>	番号なし版 <code>align</code>	<code>amsmath</code>
<code>gather</code>	複数行を中央揃え, 各行に番号	<code>amsmath</code>
<code>multline</code>	長い 1 式を複数行に折り返す	<code>amsmath</code>
<code>split</code>	<code>equation</code> 内で複数行に分割（番号は 1 つ）	<code>amsmath</code>
<code>cases</code>	場合分け（左に大括弧）	<code>amsmath</code>
<code>pmatrix</code>	括弧付き行列（丸括弧）	<code>amsmath</code>
<code>bmatrix</code>	括弧付き行列（角括弧）	<code>amsmath</code>
<code>vmatrix</code>	行列式（縦棒）	<code>amsmath</code>